The VAE Algorithm: Step by Step

This note walks through what a Variational Autoencoder actually does at each forward pass, and why each step is necessary — not just *that* it works, but the causal chain that makes it work.

Setup

For an input x, a VAE learns two networks: an **encoder** $q_\phi(z \mid x)$ that maps x to a distribution over latent codes z, and a **decoder** $p_\theta(x \mid z)$ that maps a latent code back to a reconstruction $\hat{x}$. Unlike a plain autoencoder, the encoder never outputs a single point — it outputs the parameters of a distribution.

Where the ELBO itself comes from

Before deriving the gradient problem, it's worth deriving the objective $\mathcal{L}(\theta, \phi; x)$ itself — where does this particular expression come from, and why is it called a "lower bound"?

The starting point: maximize the data likelihood

What we actually want to train the model to do is assign high probability to the real data — maximize $\log p_\theta(x)$, the log-likelihood of x under the generative model. But $p_\theta(x)$ is defined by marginalizing out the latent variable z:

$$p_\theta(x) = \int p_\theta(x, z)\, dz = \int p_\theta(x \mid z)\, p(z)\, dz$$

This integral is intractable in general — z is continuous, and $p_\theta(x \mid z)$ is a neural network, so there is no closed form. $\log p_\theta(x)$ cannot be computed or differentiated directly. This is the actual reason a VAE needs an encoder $q_\phi(z \mid x)$ at all: it exists to make this intractable quantity approachable.

Introducing $q_\phi(z \mid x)$

Multiply and divide the integrand by an arbitrary distribution $q_\phi(z \mid x)$ (this will become the encoder) — a standard trick called *importance sampling*:

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz = \int q_\phi(z \mid x) \cdot \frac{p_\theta(x \mid z)\, p(z)}{q_\phi(z \mid x)}\, dz = \mathbb{E}_{z \sim q_\phi(z|x)} \left[\frac{p_\theta(x, z)}{q_\phi(z \mid x)} \right]$$

Taking log of both sides:

$$\log p_\theta(x) = \log\, \mathbb{E}_{z \sim q_\phi(z|x)} \left[\frac{p_\theta(x, z)}{q_\phi(z \mid x)} \right]$$

This is still exact, and still intractable to compute directly — the log sits *outside* the expectation, and $\log \mathbb{E}[\cdot] \neq \mathbb{E}[\log(\cdot)]$ in general.

Jensen's inequality: turning this into a bound

log is a concave function, so Jensen's inequality applies: for any concave g, $g(\mathbb{E}[X]) \geq \mathbb{E}[g(X)]$. Applying this with $g = \log$:

$$\log p_\theta(x) = \log \mathbb{E}_{q_\phi(z|x)}\left[\frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right] \quad \geq \quad \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right]$$

The right-hand side — call it $\mathcal{L}(\theta, \phi; x)$ — is now a genuine **lower bound** on the log-likelihood we actually want to maximize. This is the Evidence Lower BOund (ELBO): "evidence" is another name for $\log p_\theta(x)$, and this quantity lower-bounds it. Since log and $\mathbb{E}$ can be swapped here, this expression *is* tractable — it only requires being able to sample from $q_\phi(z \mid x)$ and evaluate the resulting log-densities, which is exactly what the encoder provides.

Expanding the bound into the reconstruction + KL form

Expand $p_\theta(x, z) = p_\theta(x \mid z)\, p(z)$ inside the bound and split the log of a product into a sum:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x \mid z)\, p(z)}{q_\phi(z \mid x)}\right] = \mathbb{E}_{q_\phi(z|x)}\Big[\log p_\theta(x \mid z)\Big] + \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p(z)}{q_\phi(z \mid x)}\right]$$

The second term is, by definition, the negative KL divergence between $q_\phi(z \mid x)$ and the prior $p(z)$:

$$\mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p(z)}{q_\phi(z \mid x)}\right] = -\,\mathbb{E}_{q_\phi(z|x)}\left[\log \frac{q_\phi(z \mid x)}{p(z)}\right] = -\,D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

Substituting back gives exactly the expression at the top of this note:

$$\boxed{\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{z \sim q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] \;-\; D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}$$

The first term rewards z's sampled from the encoder that let the decoder reconstruct x well — the **reconstruction term**. The second term penalizes the encoder's distribution for straying from the prior $p(z) = \mathcal{N}(0, I)$ — the **KL term** used throughout Step 5 below.

How tight is the bound?

Subtracting the ELBO from the true log-likelihood recovers exactly the KL divergence between the encoder's posterior $q_\phi(z \mid x)$ and the model's true (but intractable) posterior $p_\theta(z \mid x)$:

$$\log p_\theta(x) - \mathcal{L}(\theta, \phi; x) = D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big) \;\geq\; 0$$

Since a KL divergence is always ≥ 0, this confirms $\mathcal{L}(\theta, \phi; x) \leq \log p_\theta(x)$ — the bound is genuinely a *lower* bound, with equality only when the encoder's approximate posterior exactly matches the true posterior. This is also why training pushes $q_\phi(z \mid x)$ to become as good an approximation to $p_\theta(z \mid x)$ as its Gaussian form allows: maximizing the ELBO with respect to ϕ simultaneously tightens this gap.

From the ELBO to an actual loss function

The ELBO derived above is something to be *maximized*. Deep learning libraries are built around *minimizing* a loss, so the first step is trivial — negate it:

$$\mathcal{L}_{\text{loss}}(\theta, \phi; x) = -\mathcal{L}(\theta, \phi; x) = \underbrace{-\,\mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big]}_{\text{reconstruction loss}} + \underbrace{D_{\text{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\text{KL loss}}$$

The KL term already has the closed form derived in Step 5. What still needs deriving is the reconstruction term — and it needs two separate things resolved, not one: (a) how the expectation $\mathbb{E}_{q_\phi(z|x)}[\cdot]$ disappears, and (b) what $-\log p_\theta(x \mid z)$ actually evaluates to once a specific z is plugged in.

Step (a): getting rid of the expectation

$\mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big]$ is, in full, an integral over every possible z:

$$\mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] = \int q_\phi(z \mid x)\, \log p_\theta(x \mid z)\, dz$$

This is exactly as intractable as the original $p_\theta(x)$ integral from the ELBO derivation — no closed form exists once $p_\theta(x \mid z)$ is a neural network. It is not solved analytically; it is estimated with **Monte Carlo sampling**. Draw one sample $z^{(1)}$ from $q_\phi(z \mid x)$ using the reparameterization from Step 2/4,

$$z^{(1)} = \mu + \sigma \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

and use $\log p_\theta(x \mid z^{(1)})$ — the value at that single sampled point — as an estimate of the whole integral:

$$\mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] \;\approx\; \log p_\theta\big(x \mid z^{(1)}\big)$$

This is a biased-looking move, but it is a standard, *unbiased* Monte Carlo estimator: in expectation over many draws of ε, $\log p_\theta(x \mid z^{(1)})$ averages out to the true integral. A single sample per forward pass is enough because training re-draws a fresh ε on every one of many batches and epochs — the noise in any one estimate washes out over the course of training, the same way single-sample stochastic gradient descent works even though each individual gradient is a noisy estimate of the true gradient over the full dataset. This is precisely why ε needed to be reparameterized in the first place: the estimator above is only usable for training if it can also be differentiated with respect to ϕ, which requires $z^{(1)}$ to be a differentiable function of ϕ rather than an opaque stochastic draw.

Once this substitution is made, the expectation is gone, and $-\log p_\theta(x \mid z^{(1)})$ is an ordinary deterministic scalar — a plain function of x and the network's output at the sampled $z^{(1)}$. *That* is the quantity Cases 1 and 2 below actually derive a closed form for.

Step (b): what $-\log p_\theta(x \mid z)$ evaluates to

With the expectation replaced by a single evaluation at $z = z^{(1)}$, the reconstruction loss for one data point is $-\log p_\theta(x \mid z^{(1)})$ — a concrete number once you commit to a distributional form for $p_\theta(x \mid z)$.

Case 1: Bernoulli decoder → binary cross-entropy

If each output pixel $x_i \in [0, 1]$ (e.g. normalized MNIST pixels) is modeled as an independent Bernoulli random variable, with the decoder outputting the probability $\hat{x}_i = p_\theta(x_i = 1 \mid z)$ for each pixel, the likelihood of the full output x given z factorizes:

$$p_\theta(x \mid z) = \prod_{i=1}^{d} \hat{x}_i^{x_i} \, (1 - \hat{x}_i)^{\,1 - x_i}$$

Taking log turns the product into a sum:

$$\log p_\theta(x \mid z) = \sum_{i=1}^{d} \Big[\, x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i) \, \Big]$$

Negating gives exactly the standard binary cross-entropy loss, summed over pixels:

$$-\log p_\theta(x \mid z) = -\sum_{i=1}^{d} \Big[\, x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i) \, \Big] = \mathrm{BCE}(x, \hat{x})$$

This is why an MNIST-style VAE decoder ends in a sigmoid (to produce valid probabilities $\hat{x}_i \in [0, 1]$) and is trained with `BCELoss` or `binary_cross_entropy` — that loss *is* $-\log p_\theta(x \mid z)$ under a Bernoulli assumption, not an arbitrary choice of loss function.

Case 2: Gaussian decoder → mean squared error

If instead each output dimension is modeled as Gaussian with the decoder producing the mean $\hat{x} = f_\theta(z)$ and a fixed, isotropic variance σ^2 (a common simplification — the decoder is not trained to predict variance at all):

$$p_\theta(x \mid z) = \mathcal{N}(x;\, \hat{x},\, \sigma^2 I) = \frac{1}{(2\pi\sigma^2)^{d/2}} \exp\left(-\frac{\|x - \hat{x}\|^2}{2\sigma^2} \right)$$

Taking log:

$$\log p_\theta(x \mid z) = -\frac{\|x - \hat{x}\|^2}{2\sigma^2} - \frac{d}{2} \log(2\pi\sigma^2)$$

The second term does not depend on $\hat{x}$, so it contributes nothing to the gradient with respect to θ or ϕ and can be dropped as a constant. Negating what remains:

$$-\log p_\theta(x \mid z) \;\propto\; \|x - \hat{x}\|^2 = \mathrm{MSE}(x, \hat{x}) \quad \text{(up to the constant scale factor } \tfrac{1}{2\sigma^2}\text{)}$$

This is why the swiss-roll VAE — continuous-valued, unbounded, standardized 2D coordinates, not pixel probabilities — is trained with `MSELoss` instead of BCE: the data does not fit a Bernoulli assumption, but does fit a Gaussian observation-noise assumption around the decoder's output.

The general pattern

In both cases the recipe is the same: *pick a likelihood family for $p_\theta(x \mid z)$ that matches what the data actually looks like, then $-\log$ of that likelihood, evaluated at the single reparameterized sample $z^{(1)}$, is the reconstruction loss* — not a separately chosen heuristic. Bernoulli data (bounded, probabilistic, e.g. pixel-on/off) gives BCE. Gaussian data (continuous, unbounded, real-valued) gives MSE.

The loss actually minimized in code

Putting both terms together, per data point x:

$$\mathcal{L}_{\text{loss}} = \underbrace{\text{BCE}(x, \hat{x}) \;\text{ or }\; \text{MSE}(x, \hat{x})}_{\text{reconstruction, from the decoder's likelihood family}} + \underbrace{\left(-\frac{1}{2} \sum_{i=1}^{d} \left(1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2\right) \right)}_{\text{KL, from Step 5}}$$

This is the single scalar that `loss.backward()` is called on. Minimizing it via gradient descent is, by construction, equivalent to maximizing the ELBO $\mathcal{L}(\theta, \phi; x)$ derived above — which is in turn a lower bound on the true data log-likelihood $\log p_\theta(x)$ that the model is ultimately trying to maximize.

Where reparameterization comes from: the gradient problem

Before the step-by-step, it is worth deriving *why* the reparameterization trick is needed at all, starting from the raw training objective — this is the problem Step 4 below solves.

The objective

A VAE is trained to maximize the evidence lower bound (ELBO) with respect to both the decoder parameters θ and the encoder parameters ϕ:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{z \sim q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] \;-\; D_{\text{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

The KL term has a closed form (derived in Step 5) and is not the problem. The difficulty is the first term — an expectation of some function $f(z) = \log p_\theta(x \mid z)$ under a distribution $q_\phi(z \mid x)$ that itself depends on ϕ:

$$\mathbb{E}_{z \sim q_\phi(z|x)}\big[f(z)\big] = \int f(z)\, q_\phi(z \mid x)\, dz$$

To train the encoder with gradient descent, this term needs ∇_ϕ.

Why you can't just differentiate through it

Naively pushing the gradient inside the integral (valid here, since the integration domain does not depend on ϕ) gives:

$$\nabla_\phi\, \mathbb{E}_{z \sim q_\phi(z|x)}\big[f(z)\big] = \int f(z)\, \nabla_\phi\, q_\phi(z \mid x)\, dz$$

This is a well-defined integral, but it is no longer written as an expectation under q_ϕ — $\nabla_\phi\, q_\phi(z \mid x)$ is not itself a probability density (it can be negative), so there is no way to estimate it by drawing

samples $z \sim q_\phi(z \mid x)$ and averaging, the way ordinary Monte Carlo estimation works. In short: you can sample z from the distribution, but you cannot backpropagate through the act of sampling itself, because the randomness and the parameters ϕ are entangled inside the same stochastic node.

The fix: change of variables

The trick is to rewrite z as a deterministic, differentiable function of ϕ and an auxiliary noise variable ε drawn from a *fixed* distribution that does not depend on ϕ:

$$z = g(\varepsilon; \phi), \qquad \varepsilon \sim p(\varepsilon)$$

For a diagonal Gaussian posterior $q_\phi(z \mid x) = \mathcal{N}(\mu, \sigma^2)$, this transformation is:

$$g(\varepsilon; \phi) = \mu + \sigma \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

This is a valid change of variables — sampling $\varepsilon \sim \mathcal{N}(0, I)$ and computing $z = \mu + \sigma\varepsilon$ produces exactly the same distribution over z as sampling $z \sim \mathcal{N}(\mu, \sigma^2)$ directly. So the expectation can be rewritten with ε, not z, as the random variable being integrated over:

$$\mathbb{E}_{z \sim q_\phi(z|x)}\big[f(z)\big] = \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,I)}\big[f(g(\varepsilon; \phi))\big]$$

Now ϕ appears *only* inside the function being averaged, and the distribution being sampled from, $\mathcal{N}(0, I)$, is fixed and independent of ϕ. That means the gradient can be pushed inside the expectation legitimately:

$$\nabla_\phi \, \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,I)}\big[f(g(\varepsilon; \phi))\big] = \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,I)}\big[\nabla_\phi \, f(g(\varepsilon; \phi))\big]$$

The right-hand side is an ordinary expectation of a gradient, estimable with a single Monte Carlo sample of ε — and since g is just elementwise multiplication and addition, $\nabla_\phi f(g(\varepsilon; \phi))$ is computed by plain backpropagation through g, then through the encoder network that produced μ and σ. This is precisely what Step 4 below implements.

1 The encoder outputs a distribution, not a point

For input x, the encoder network produces two vectors, μ and $\log \sigma^2$ (written `logvar` in code), describing a diagonal Gaussian:

$$q_\phi(z \mid x) = \mathcal{N}(\mu, \sigma^2), \qquad \sigma = \exp\left(\tfrac{1}{2} \log \sigma^2\right)$$

The log-variance parameterization is used instead of predicting σ directly because it lets the network output any real number while still guaranteeing $\sigma > 0$ after the exponential — no constrained activation function needed.

This says: *"the latent code for this x is probably around μ, with uncertainty σ"* — not *"the latent code is exactly this one number."* Every input is mapped to a small cloud of plausible codes, not a pin-point location.

2 Sample from that distribution

The actual code fed to the decoder is a single sample drawn from that Gaussian:

$$z = \mu + \sigma \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

where ε is fresh random noise drawn independently on every forward pass, and $\odot$ denotes element-wise multiplication. This means z is *different every time* the same x is passed through the network during training, because ε changes each time — only μ and σ are deterministic functions of x.

3 Why sample at all, instead of decoding μ directly?

Because during training, the decoder needs to see not just the "ideal" center point μ, but also nearby noisy variants of it — and still reconstruct x well from all of them.

If the decoder is trained so that μ *and* a spread of nearby samples all reconstruct correctly, then the entire neighborhood around μ becomes meaningful and decodable, not just that one exact point. This is what makes the latent space **smooth and continuous** rather than a set of isolated, memorized spikes — a property a plain autoencoder's latent space has no reason to have, since a plain AE never forces the decoder to tolerate perturbed inputs.

4 The reparameterization trick, applied

The derivation above (*Where reparameterization comes from*) showed the general problem — a gradient with respect to ϕ cannot pass through a sampling node whose own distribution depends on ϕ — and its fix, the change of variables $z = g(\varepsilon; \phi)$ with ε drawn from a fixed, ϕ-independent distribution. Concretely, for the VAE's diagonal Gaussian posterior, that transformation is:

$$z = \mu + \sigma \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I) \text{ sampled fresh, independent of the network}$$

z is now a deterministic, differentiable function of μ, σ, and the externally-sampled constant ε:

$$\frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial \log \sigma^2} = \frac{1}{2}\, \sigma \odot \varepsilon$$

Gradients from the reconstruction loss can now flow backward: loss $\to \hat{x} \to z \to (\mu, \log \sigma^2) \to$ encoder weights. The encoder trains with ordinary backpropagation, exactly as if there were no randomness involved at all.

In code, this is the `reparameterize` function:

```
def reparameterize(mu, logvar):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    return mu + std * eps
```

5 The KL divergence term

What it penalizes

Without any constraint, the encoder has degenerate solutions available to it. It could:

- shrink $\sigma \to 0$ for every input, eliminating the injected noise entirely and reducing to ordinary (non-variational) autoencoder behavior, or
- push μ far apart for different inputs, spreading codes out so widely that their small noisy clouds never overlap.

Both would make reconstruction easy but would destroy the smooth, generatable latent space the sampling step was meant to build.

The fix

Add a penalty term to the loss that pulls every input's distribution toward the standard normal prior $\mathcal{N}(0, I)$. For a diagonal Gaussian posterior, this has a closed form:

$$D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\big) = -\frac{1}{2} \sum_{i=1}^{d} \big(1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2\big)$$

Written in `logvar` form (as in code), with $\sigma_i^2 = \exp(\log \sigma_i^2)$:

$$\mathcal{L}_{\mathrm{KL}} = -\frac{1}{2} \sum_{i=1}^{d} \Big(1 + \texttt{logvar}_i - \mu_i^2 - \exp(\texttt{logvar}_i)\Big)$$

This term is minimized (equals zero) exactly when $\mu_i = 0$ and $\sigma_i^2 = 1$ for every dimension i — i.e. when the posterior matches the prior exactly.

The full VAE loss

$$\mathcal{L}_{\mathrm{VAE}} = \underbrace{\mathbb{E}_{q_\phi(z|x)}\big[-\log p_\theta(x \mid z)\big]}_{\text{reconstruction term}} + \underbrace{D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\big)}_{\text{KL term}}$$

This is the (negative) Evidence Lower Bound, or ELBO. In practice the reconstruction term is approximated with a single sample of z (the reparameterized sample from Step 2), and implemented as MSE or BCE loss between $\hat{x}$ and x depending on the data type.

The resulting tension

- **Reconstruction** wants codes that are spread out and low-noise, so different inputs are easy to tell apart and decode accurately.
- **KL** wants every code pulled toward the same fixed distribution, $\mathcal{N}(0, I)$, regardless of input.

Training settles at a compromise: codes spread out *just enough* to remain distinguishable, while still overlapping enough that the latent space stays densely, continuously populated. This is also why the KL loss typically *increases* over the course of training even as total loss falls — the reconstruction term initially dominates the improvement, pulling codes apart, and KL is dragged upward as a side effect until the two terms reach equilibrium.

6 Inference-time behavior: switch back to μ

At inference or analysis time — inspecting the latent space, plotting a learned curve, running a downstream classifier on the codes — the sampling step is dropped. Use μ directly, with no added noise:

$$z_{\text{inference}} = \mu \qquad (\text{no } \varepsilon \text{ term})$$

The injected noise exists purely to serve the training-time purpose in Step 3 (forcing the decoder to tolerate a neighborhood, not just a point). Once training is done, μ is the network's deterministic best estimate of where x belongs in latent space, and reintroducing noise at inspection time would only add uninformative jitter.

7 Generating new samples

Because the KL term explicitly regularizes $q_\phi(z \mid x)$ toward $\mathcal{N}(0, I)$ for every training point, the *aggregate* latent space is trained to resemble a standard normal distribution as well. This means new data can be generated directly from the prior, without needing any real input x:

$$z \sim \mathcal{N}(0, I) \quad \longrightarrow \quad \hat{x} = \text{decoder}(z)$$

This is not available "for free" in a plain autoencoder — there, the latent space has no defined shape to sample from, so a random z typically decodes to nonsense.

Summary: the causal chain

1. The encoder outputs a distribution (μ, σ), not a point.
2. A sample $z = \mu + \sigma\varepsilon$ is drawn and decoded — different each pass.
3. This forces the decoder to reconstruct correctly not just at μ, but across a neighborhood around it $\Rightarrow$ nearby latent points become meaningful too.
4. The reparameterization trick makes this sampling differentiable, so the whole thing trains with ordinary backpropagation.
5. The KL term pulls every (μ, σ) toward $\mathcal{N}(0, I)$, preventing collapse (noise $\to 0$) or unbounded spreading — the tension with the reconstruction loss settles into codes that are distinguishable *and* densely packed.
6. The combined effect: the latent space becomes a dense, continuous region shaped like $\mathcal{N}(0, I)$, which can be sampled from directly to **generate new data** — something a plain autoencoder's latent space has no mechanism to support.